

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 678 — Distributed Systems and Cloud Computing

Consistency, consensus, and failure — with multiplayer netcode as the worked example

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 5 (with CSCE 620 and CSCE 605)

Prerequisites: CSCE 611 Operating Systems; CSCE 608 Database Systems; CSCE 735 Parallel Computing is helpful

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A replicated system that survives the failures you inject into it, with a stated consistency model, measured tail latency, and an honest account of what it does when the network partitions

Course description

Every other course in this program assumed one machine. The machine might have had many cores (CSCE 735) or a hostile scheduler (CSCE 611), but it was one machine: when it failed, everything failed together, and that turns out to have been an enormous simplification.

The organising fact of this course is partial failure. In a distributed system, components fail independently and *the survivors cannot tell what happened*. A request that received no reply might have been lost on the way out, executed and lost on the way back, or be executing still. **Those three cases are indistinguishable from the caller**, they demand different responses, and essentially every difficulty in this subject traces back to that one ambiguity.

The second theme is that **the impossibility results are real and are routinely misquoted**. CAP does not say pick two; FLP does not say consensus is impossible; eventual consistency is not a synonym for 'no guarantees'. **Module 05 states each result precisely** and says what it actually forbids, because the folklore versions lead to bad architecture.

The worked example is **multiplayer netcode**, which is distributed systems under the hardest constraint in the subject: **a hard real-time deadline, an actively adversarial participant, and no option to make the user wait**. A database can block for 200 ms and nobody notices. A game cannot block for 16. **Modules 09 through 11 show what you do instead**, and the answer — predict locally, reconcile against authority, and trust the client with nothing — is a genuinely elegant piece of engineering that is far less widely understood than it deserves to be.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain partial failure and why it is the central difficulty.
- 2 Reason about latency, bandwidth, and protocol choice.
- 3 Use logical clocks and explain why physical time is unreliable.
- 4 Compare consistency models and choose one deliberately.
- 5 State CAP, FLP, and PACELC precisely and say what each forbids.
- 6 Explain Raft and implement leader election and log replication.
- 7 Design failure detection and membership that tolerates false positives.
- 8 Partition data and apply CRDTs where they fit.
- 9 Implement client-side prediction and server reconciliation.
- 10 Design for an adversarial client, and operate what you built.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Partial Failure	Why is a distributed system not just a program on several machines?
2	The Network Underneath	What should you build on — and what does it cost?
3	Time, Clocks, and Ordering	What does it mean for one event to happen before another?
4	Replication and Consistency	What does a read return when there are many copies?
5	The Impossibility Results	What is provably impossible, and under what assumptions?
6	Consensus	How do several machines agree, despite failures?
7	Failure Detection and Membership	How do you tell a dead node from a slow one?
8	Distributed Data	How do you partition data, and transact across partitions?
9	Netcode I: The Latency Problem	How do you build a shared world when the network takes 100 ms?
10	Netcode II: Prediction and Reconciliation	How do you hide a 100 millisecond round trip?
11	Trust, Cheating, and Fairness	What can you believe a client when it tells you?
12	Cloud, Scaling, and Economics	When does the cloud help, and what do you pay for it?

Wk	Module	Core question it answers
13	Operating It	How do you run a distributed system and know that it works?

Projects

Project 1 — Replicated state machine (after Module 7)

Implement Raft, or an equivalent consensus protocol, and build a replicated key-value store on top of it — then break it deliberately and show that it survives.

Requirements

- **Leader election with randomised timeouts**, surviving repeated leader failure.
- Log replication with the commit rule stated and tested.
- **Persistence across restart**, so a node that crashes and returns does not violate safety.
- **A fault injector**: drop messages, delay them, reorder them, partition the cluster, and kill and restart nodes.
- **A linearizability checker** over the operation history — Porcupine or Knossos, or your own.
- A deterministic replay harness for debugging.

Done means

- **The linearizability checker passes over thousands of randomised histories with faults injected throughout**. That is the deliverable, not a demo of it working.
- **A partition test**: split the cluster, show the minority stops accepting writes, heal it, show it catches up.
- **A bug you found with the fault injector**, with the trace that revealed it. If you found none, say how many histories you checked.
- Election latency measured as a distribution across many leader failures.

Project 2 — Netcode for a real-time simulation (after Module 12)

Take a simulation — ideally your CSCE 649 physics project — and make it multiplayer over a hostile network, with an authoritative server.

Requirements

- **An authoritative server** that trusts the client for nothing except its own input.
- **Client-side prediction with server reconciliation**, including the replay of unacknowledged inputs.
- Entity interpolation for remote objects, with the buffer delay stated.
- **Measured behaviour at 50, 150, and 300 ms RTT, and at 1%, 5%, and 10% packet loss**, with jitter.
- **A cheating attempt you implemented and your server defeated** — a speedhack, a teleport, or a fabricated hit.
- Bandwidth measured per client per second.

Done means

- **A video or frame sequence at each latency and loss setting**, so the degradation is visible rather than asserted.
- **A plot of reconciliation corrections against latency** — how often the server disagreed with the prediction, and by how much.
- **The cheat demonstrated and then shown to fail**, with the server-side check that caught it.
- **An honest statement of what still feels bad**, and at what latency your approach stops working. Every netcode has that point.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
MIT 6.5840 — Distributed Systems (free lectures and labs)	The primary source. Complete video lectures, the paper list, and the Raft labs. The best free distributed systems course that exists, and the labs are the real value.
Martin Kleppmann — Designing Data-Intensive Applications	The reference for Modules 03 through 08. Unusually careful about what the guarantees actually are. Library copy; the author's lecture series is free on YouTube.
van Steen & Tanenbaum — Distributed Systems (free book)	Free in full. Broader and more systematic than Kleppmann, stronger on naming, coordination, and the formal material.
Ongaro & Ousterhout — In Search of an Understandable Consensus Algorithm (free)	The Raft paper, and the Module 06 project. Deliberately written to be implementable, and the visualisation on the same site is excellent.

Resource	Role
Gaffer On Games — networked physics and game networking (free)	The reference for Modules 09 through 11. Glenn Fiedler's series on prediction, reconciliation, and snapshot interpolation is the clearest treatment of netcode available.
Google — Site Reliability Engineering and the SRE Workbook (free)	The Module 13 material: SLOs, error budgets, tail latency, and what operating a distributed system actually involves.

Tooling

- **Go or Rust.** Both make concurrent network code tolerable, and the MIT labs are in Go. **Whatever you choose, you need good concurrency primitives**, because every module after the second is concurrent.
- **A network simulator or emulator** — tc netem on Linux, Clumsy on Windows, or your own in-process one. **You must be able to add latency, jitter, loss, and reordering on demand**; a system tested only on localhost has not been tested.
- **A deterministic test harness** that can run the whole system in one process with a controlled clock and a controlled scheduler. **Module 13 argues this is the only way to debug these systems**, and it must be designed in from the beginning.
- **Containers**, for running many nodes on one machine. Docker Compose is enough; a cluster is not required for any of this.
- **A way to plot latency distributions**, including the percentiles. **Means are useless here** and Module 13 explains why at length.
- **Wireshark or tcpdump**, for Module 02. Looking at the actual packets once is worth a great deal.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.